

PRACTICAL COURSE WORKBOOK

Applied TypeScript

Build and test a responsive digital product

LEVEL	Intermediate
GUIDED TIME	5 hours across 12 lessons
PROJECT	a responsive, accessible application you can demonstrate
SKILLS	TypeScript, Generics, Utility Types, Critical evaluation, Documentation
EDITORIAL STATUS	Structured draft v0.9

WHAT YOU WILL BE ABLE TO DO

- Use TypeScript appropriately in a realistic, bounded task.
- Use Generics appropriately in a realistic, bounded task.
- Use Utility Types appropriately in a realistic, bounded task.

WHO THIS IS FOR

Learners building accessible, maintainable web products.

BEFORE YOU BEGIN

- Comfort using a browser and text editor

This open workbook is a structured draft undergoing course-specific editorial review. It does not provide certification.

COURSE MAP

From foundation to finished work

Applied TypeScript is a structured, practical course covering TypeScript, Generics, Utility Types. It emphasizes explainable methods, checked work and a final outcome that can be reviewed.

MODULE	LESSONS AND FOCUS
01 TypeScript: Web foundations	1. Standards with TypeScript 2. Structure with Generics 3. Developer tools with Utility Types
02 Generics: Core implementation	1. Components with TypeScript 2. State and data with Generics 3. Responsive layout with Utility Types
03 Utility Types: Quality	1. Accessibility with TypeScript 2. Testing with Generics 3. Performance with Utility Types
04 TypeScript: Production practice	1. Security with TypeScript 2. Deployment with Generics 3. Maintenance with Utility Types

HOW TO STUDY EACH LESSON

- 1 Read the objective and identify the decision it supports.
- 2 Reproduce the worked example with the stated input.
- 3 Test one normal case and one boundary or failure case.
- 4 Complete the knowledge check without guessing.
- 5 Save a short evidence note: result, limitation and next action.

CAPSTONE PROJECT

Produce a reviewable Applied TypeScript project using TypeScript, Generics, Utility Types.

DELIVERABLE	A working Applied TypeScript artefact plus an evidence-based self-review.
TOOLS	A suitable TypeScript environment, A plain-text decision log, Test data or realistic sample material

Production plan

- 1 Define the intended user, outcome and constraints.
- 2 Create the smallest complete result using TypeScript.
- 3 Test one normal case, one boundary case and one failure response.
- 4 Revise the work from the evidence and preserve before-and-after results.
- 5 Prepare a concise handover containing method, limitations and next step.

Definition of done

- The output matches the stated outcome.
- Inputs and decisions are reproducible.
- Boundary and failure evidence is included.
- Limitations and responsibility considerations are explicit.

Evidence log

INPUT	Original brief, data, requirements or test material.
DECISIONS	Chosen method and one reasonable alternative.
TESTS	Normal, boundary and failure results.
LIMITATION	What the result does not establish.
REVISION	One evidence-led improvement.

REVIEW AND SOURCES

Make the work credible and repeatable

Final self-review

- Can another learner repeat the method?
- Did I test a difficult case?
- Did I avoid unsupported claims?
- Is the next action proportionate to the remaining risk?

Authoritative references

Learn web development

MDN Web Docs - reviewed 2026-08-21

https://developer.mozilla.org/en-US/docs/Learn_web_development

WCAG 2.2

W3C - reviewed 2026-08-21

<https://www.w3.org/TR/WCAG22/>

SOURCE NOTE

Product versions, laws and professional standards can change. Recheck the linked primary source before applying version-sensitive information.

NEXT IMPROVEMENT

Choose one weakness found during review and improve it before extending the TypeScript scope.